AgentDescent: Gradient Descent Where the Parameters Are Agents

A Parallel, Asynchronous, Merge-Based Framework for Self-Evolving Agent Artifacts

Preprint

Danyang Chen
birfychen@gmail.com
https://github.com/Birfy/agentdescent

August 2026

Abstract

Self-improving agent systems evolve their own skills, prompts, and harnesses, but largely through a *serial* loop that bounds improvement throughput at $O(1/T_{\text{iter}})$. **AgentDescent** transplants the distributed-training stack onto this problem: artifacts as parameters, diffs with attached evidence as gradients, a version-vectored ledger as the parameter server, and merge decisions as optimizer steps. Because diffs do not add, aggregation becomes a discrete-space optimization — conflict resolution, fusion, and statistical acceptance under a per-artifact Beta posterior. Eighteen published algorithms port onto the engine as plug-ins and run unchanged under serial, synchronous, and barrier-free scheduling.

Merging supplies a source of speedup that selection cannot: a round's k concurrent proposals are validated once rather than k times, and on live models validation is the critical path. That saving has a precondition — the proposals must be fusable — and it fails exactly where an artifact is held in one key, since every concurrent pair then conflicts and merge-of-N degenerates into best-of-N selection at best-of-N validation cost. *Reflective merge* closes that gap by having a model write the union of contradicting proposals, which then faces the same acceptance test as any candidate. We isolate it: on a one-key artifact, the keyed union fuses in 0 of 48 merges while reflective merge fuses in 42, and it spends 47% fewer model calls at a pinned rollout budget (paired 95% CI [−634, −311] calls), with no quality difference detectable at four seeds. We also derive when merging can fire at all, in closed form. Measured on live models throughout: parallel execution reaches 6.8× less wall-clock than a faithful serial control, and eighteen published algorithms run on the engine as plug-ins. Code, workloads, and per-result reproduction commands are open source.

Keywords Self-evolving agents · Parallel and asynchronous execution · Diff merging · LLM agents · Distributed optimization

1 Introduction

A growing body of work shows that large-language-model agents can improve their own artifacts: prompts [4, 6], in-context playbooks [39], skill libraries [30, 41], agentic workflows [9, 38], and their own code [23, 34, 37]. Most of these systems share an execution structure inherited from the Gödel machine [25]: a serial improvement loop in which one candidate modification is proposed, evaluated, and accepted or rejected before the next is considered. If one iteration takes T_{iter} wall-clock seconds, improvement throughput is bounded at one accepted change per T_{iter}, independent of available compute. Notable exceptions parallelize at the *population* level — FunSearch and AlphaEvolve run asynchronous samplers over island archives [22, 24] — but their candidates remain independent: concurrent work is selected among, not merged.

Distributed deep learning encountered the same bottleneck and resolved it with a now-standard stack: data-parallel workers computing gradients concurrently, parameter servers aggregating them [17], staleness-tolerant asynchronous updates [21], per-parameter adaptive steps [15], and gradient surgery for conflicting updates [35]. This paper develops the thesis that the same stack, suitably reinterpreted, applies to agent self-evolution — and identifies precisely where the reinterpretation must depart from the original.

AgentDescent (Fig. 1) instantiates the mapping. A library of evolvable artifacts plays the role of the parameter tensor; a diff annotated with an *evidence card* (the rollout that motivated it) plays the role of a gradient; a git-backed, version-vectored ledger plays the role of the parameter server; and the optimizer step is an *aggregator* that merges concurrent diffs. N workers roll out tasks against snapshots of the library and propose diffs in parallel; an asynchronous merger aggregates them, targeting $O(N/T_{\text{iter}})$ improvement throughput.

The mapping is productive precisely because of where it breaks. *Gradients add; diffs do not.* Two gradients can always be summed; two textual edits to the same artifact may be complementary, redundant, or contradictory. Aggregation is therefore not averaging but a decision procedure — staleness filtering, conflict resolution, fusion, and a statistical acceptance test (Section 4). A second disanalogy is equally consequential: training code is not modifiable by the gradients it computes, whereas a self-evolving system can in principle rewrite its own verifier. AgentDescent therefore enforces a layered governance model in which safety-critical components are frozen (Section 6).

Contributions.

1. **A system design and a discrete-space aggregator** mapping the distributed-training stack onto parallel self-evolution — per-diff bounded staleness, conflict projection, fusion, Beta-posterior acceptance, dual-branch promotion — with a formal account of where the analogy holds and where it must break (Sections 3 and 4).
2. **An efficiency analysis identifying three sources of speedup** (Section 5): rollout overlap, barrier removal, and — specific to merge-based evolution — reduced validation cost, in which merging collapses k acceptance tests into one and *reflective merge* extends the reduction to artifacts whose concurrent proposals conflict. The three do not compose unconditionally, and we characterize the regimes in which each pays.
3. **An extensibility mechanism** under which published algorithms are ported as plug-ins rather than forks (Sections 3.3 and 7): a diff-encoding `strategy` plus an optional `aggregator_factory`, or a declarative policy bundle. Eighteen ports run under all three schedulers without modification; eleven of them carry measurements in this paper and the fidelity of each is declared per port.
4. **A precondition for merging, in closed form** (Eq. (6)): because the engine proposes only from failed rollouts, the fraction of rounds with two diffs to fuse is at most $1-(1-f)^N - Nf(1-f)^{N-1}$ in the incumbent's failure rate f and the worker count N. As an upper bound it is loose — measured, it overpredicts by 2.4–6.8× — but in the direction that matters it is decisive: where it is small, merging cannot fire at all. It retrodicts two of our null results, prescribes the workload that avoids them, and bounds fusion width from below where the trust region and the round count bound it from above.
5. **A live-model evaluation** (Section 8) whose headline is an ablation of the mechanism itself (Section 8.2): on an artifact where reflective merge is the only path to a fused candidate, it fuses 42 of 48 merges against the keyed union's 0, and cuts model calls by 47% at a pinned rollout budget. Around it: a parallelism ladder against a faithful serial control (6.8× wall-clock, three seeds), quality preserved across ten of eleven ports under the most aggressive schedule, and an equal-budget merge-versus-selection comparison on a multi-key artifact.

2 Related Work

Self-evolving agents span prompt evolution [4, 6], context and playbook evolution [39], skill libraries [30, 41], workflow and harness search [9, 38], self-modifying code [23, 34, 37], evolutionary program search [16, 20, 22, 24, 26], and label-free self-play [2, 10, 40], with reflection [19, 27] as a shared inner proposal mechanism. Prompt optimization more broadly includes meta-prompting and search methods — APE [42], OPRO [32], DSPy [14], EvoPrompt [8] — and TextGrad [36], which also frames LLM feedback as a gradient analogue: TextGrad backpropagates textual critiques through a single computation graph, whereas AgentDescent's "gradients" are concurrent keyed diffs merged into one lineage under statistical acceptance. Population-based training [12] and the island architectures of FunSearch and AlphaEvolve parallelize by maintaining independent candidates and selecting among them; AgentDescent instead merges concurrent edits at the diff level. Most of the systems above publish a serial outer loop; AgentDescent treats that loop as one scheduling regime among three. On the systems side, AgentDescent borrows its vocabulary from distributed training: parameter servers [17], lock-free and bounded-staleness asynchrony [1, 7, 21], per-parameter adaptivity [15],

Table 1: The correspondence between distributed training and AgentDescent. The final row is a deliberate disanalogy, enforced rather than elided. Rows marked † are vocabulary rather than mechanism and we flag them as such: what varies per artifact is the accept/reject *threshold*, not a step magnitude, so the Adam row names a Bayesian-bandit-style conservatism; and promotion after K regression-free rounds is checkpoint promotion with hysteresis, in which no averaging occurs. The rows that carry real mechanism are the ledger, bounded staleness, and the frozen L0 layer.

Model training	**AgentDescent**
parameter tensor θ	library of `Evolvable` artifacts
gradient g	`Diff` + `EvidenceCard`
parameter server	git-backed, version-vectored `Ledger`
optimizer step	`Aggregator` merge decision
per-parameter adaptive step (Adam)†	per-artifact Beta-posterior acceptance
bounded staleness (async SGD/RL)	per-diff lag η + rebase re-verification
EMA / weight averaging†	`dev/stable` dual branch
training code (not self-modifiable)	L0 frozen governance layer

gradient surgery [35], weight averaging [31], and tensor/pipeline parallelism [11, 28]. Concurrent work on parallel self-evolution (barrier-free evolution loops; co-evolutionary verification [3]) indicates that the throughput premise is an open hypothesis; our contribution is the specific synthesis — concurrent, staleness-bounded, conflict-resolved diff-level merge over a versioned ledger — together with the measurement harness required to evaluate it.

3 Problem Setup and System Design

3.1 Problem formulation

Let $\mathcal{A}$ denote the space of *artifact libraries*: finite keyed collections $A = \{a_k\}$ of versioned textual or executable artifacts (skills, prompts, playbooks, harness modules, file trees). An agent parameterized by A induces a policy π_A; given a task distribution $\mathcal{D}$ and a bounded reward $r \in [0, 1]$, the objective is

$$A^\star = \arg\max_{A \in \mathcal{A}} \mathbb{E}_{x\sim\mathcal{D}}[\, r(\pi_A(x), x)\,] , \tag{1}$$

optimized from rollout evidence only: $\mathcal{A}$ is discrete, and no gradient of r with respect to A exists. The serial baseline — propose one candidate A', evaluate on a held-out split D_{val}, accept if better — performs one accepted modification per iteration time T_{iter}. AgentDescent's premise is that with N concurrent proposers and merge-based aggregation, accepted-modification throughput can approach $O(N/T_{\text{iter}})$ without sacrificing the acceptance discipline.

A *diff* d is a keyed partial edit over A, produced from a rollout by a *strategy* S that maps a natural-language proposal to operations on keys (rule slots, playbook bullets, file paths). The key structure is what makes two diffs comparable: edits on disjoint keys *fuse*; edits on the same key *conflict*. Every diff carries an *evidence card* $e = (x, \tau, r, v_{\text{base}})$ recording the task, rollout, reward, and the library version the proposer observed.

3.2 System components

Table 1 summarizes the correspondence and Fig. 1 the resulting architecture. The **ledger** is the parameter server: a git-backed store with per-artifact version vectors, compare-and-swap (CAS) commits, and a dual `dev/stable` branch pair; `stable` is promoted after K regression-free rounds on `dev`, an EMA-style confirmation in which a new commit restarts the counter. **Workers** hold snapshots of the library, roll out tasks, and emit (diff, evidence) pairs into a thread-safe buffer. The **aggregator** consumes the buffer and performs merge decisions (Section 4). Version vectors make the staleness of any diff computable at merge time:

$$\eta(d) = \max_{a \in \text{touched}(d)} \big(v_{\text{head}}(a) - v_{\text{base}}(a)\big). \tag{2}$$

3.3 Pluggable seams

Each component in Fig. 1 sits behind an explicit interface, selected by one keyword argument of the single entry point `evolve()`: the *agent layer* (any `prompt → text` callable is a completion — Claude, OpenAI-compatible endpoints, a tool-using CLI agent, or plain `run/reward/propose` functions), the *strategy* (diff encoding), the *aggregator* (`aggregator_factory`), the *staleness policy*, the *runtime* (synchronous or barrier-free), the *parallelism scheme* (data/tensor/pipeline [11, 28]; only the data-parallel path is exercised by any experiment here), and the *executor and*

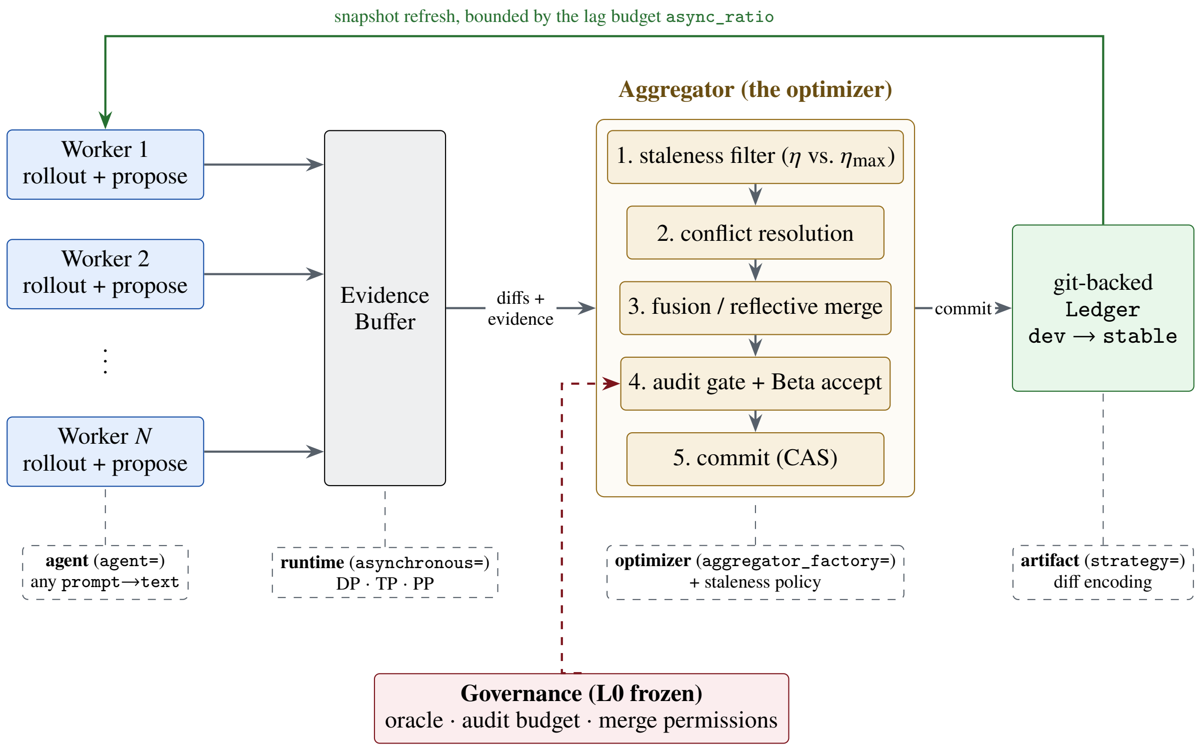

Figure 1: AgentDescent architecture. **Solid, top:** the fixed data path — N workers roll out tasks against ledger snapshots and emit diffs with evidence cards into a thread-safe buffer; a dedicated aggregator thread runs the five-stage merge pipeline and commits winners to the git-backed ledger; the green return edge is the only feedback path, bounded by the lag budget. **Dashed, bottom:** the pluggable seams of Section 3.3, each selected by one keyword argument of `evolve()`. **Red:** the frozen L0 governance layer, deliberately not a seam.

sandbox. Selection and acceptance rules are likewise named policy classes. Section 7 uses these seams to port eighteen published algorithms without modifying the engine. The single deliberate exception is governance (Section 6): the L0 layer exposes no seam, since a self-modifying system must not be able to replace its own evaluator.

4 The Aggregator as a Discrete-Space Optimizer

Evidence cards are bucketed per artifact; when a bucket reaches batch size B (or a timeout, so cold artifacts are not starved), the aggregator executes one optimizer step, summarized in Algorithm 1: **(1) staleness filtering** by Eq. (2) under the active policy (Section 5.2); **(2) conflict resolution** — syntactic (overlapping hunks) and semantic (contradictory operations) conflicts are detected, and contradictions are resolved by *dropping* the weaker member of each pair on a shared held-out subset. We name this conflict-drop rather than projection: PCGrad [35] exists precisely to *avoid* discarding a conflicting update, projecting it onto the normal plane so its non-conflicting component survives. Retaining the loser's non-conflicting keys would be the faithful analogue and is not implemented; **(3) fusion** — complementary survivors are merged into one union candidate — a union over disjoint keys, closer to disjoint task-vector merging than to the weight averaging of model soups [31], since nothing here is averaged, with *reflective merge* handling contradictions that a keyed union cannot (Section 5.3); **(4) auditing** — merges with high blast radius or low trust are routed through an oracle with veto power; **(5) statistical acceptance and commit**. Let (s_c, f_c) and (s_b, f_b) be the candidate's and the incumbent's successes and failures on the held-out split D_{val}. The candidate commits (by CAS on `dev`) iff

$$\Pr[\theta_c > \theta_b] \;>\; 1-\delta_v \quad \textbf{and} \quad \hat{r}_c \ge \hat{r}_b, \qquad \theta_b \sim \mathrm{Beta}(1+\pi_s+s_b,\; 1+\pi_f+f_b), \tag{3}$$

$$\theta_c \sim \mathrm{Beta}\Big(1+\pi_s+s_c+\textstyle\sum_j w_j \mathbb{1}[\delta_j>0],\;\; 1+\pi_f+f_c+\sum_j w_j \mathbb{1}[\delta_j \le 0]\Big), \qquad w_j = \min(1, 4|\delta_j|). \tag{4}$$

Three details of the released implementation are easy to elide and we state them because each is load-bearing. First, (π_s, π_f) is a *shared, running, per-artifact* prior rather than a uniform one: it accumulates over the run, absorbing each rejected candidate's observed delta and each commit as strong positive evidence, which is what makes the test conservative on artifacts with a history of failed edits. Second, the sum in Eq. (4) is applied to the *candidate only*: each contributing evidence card folds its own before/after delta δ_j in as a pseudo-count, so a proposal that measurably helped its own rollout enters the gate with more weight than one that did not. This is an asymmetry in the candidate's favour and it has no counterpart on the incumbent. Third, the posterior test is **and**ed with a hard point threshold: a candidate whose full held-out rate $\hat{r}_c$ falls below the incumbent's is rejected outright regardless of the posterior. On the small held-out splits used here the point threshold, not the posterior, is frequently the binding constraint — so the mechanism is a posterior test *guarded by* a regression check, not a posterior test instead of a threshold. The probability is estimated by Monte Carlo. Both posteriors are sampled independently even though s_b and s_c come from evaluating the two artifacts on the *same* D_{val}; a paired test would use that pairing and we do not, which makes the comparison noisier than it needs to be. The confidence requirement δ_v is annealed over the artifact's version v (the learning-rate-decay analogue) and a trust region caps diff size.

One risk of this discipline deserves explicit statement: the gate reuses a single fixed D_{val} across many acceptance decisions (hundreds of proposals per run in Section 8.3), so accepted diffs are selected extreme statistics on D_{val} and their measured gains are optimistically biased — the standard adaptive-data-analysis concern. Two mitigations are in place and one is not: all quality results in Section 8 are reported on a disjoint *test* split the gate never sees; the trust region and annealed δ_v limit how much any single decision can exploit D_{val}; but no reusable-holdout or D_{val}-rotation mechanism is implemented. We therefore measure the resulting bias directly (Section 8.3): across 33 runs the gain the gate observed on D_{val} exceeds the gain realized on test by 0.038 on average, a small but statistically detectable optimism that a reader should subtract from any D_{val}-reported figure.

A natural objection to merge-based evolution is that two individually beneficial edits may be jointly harmful. Two properties bound the risk: the acceptance test Eq. (3) scores the *fused* candidate on the full held-out split, so a harmful fusion cannot commit; and the union is a superset of the diffs that survive conflict resolution — which drops the weaker member of each contradicting pair first, so it is not a superset of every proposal. An optional *fusion tournament* additionally ranks the union against its constituents; it is disabled by default (it costs one additional held-out sweep per candidate against a recoverable gain) but serves as the measurement instrument for the objection, reporting win rate, the losing tail, and a `contested` counter that records whether a fused candidate was in fact constructed and ranked — so a run in which fusion never occurred cannot be reported as a merge win. Section 8.4 reports the win rate in situ for one workload; we attach no headline to it, because it is a property of that artifact's key granularity and proposal overlap rather than of the mechanism.

5 Efficiency Analysis: Three Sources of Speedup

Wall-clock time in a self-evolution loop decomposes into rollout time, synchronization time, and held-out validation time. AgentDescent addresses each with a distinct mechanism; the third is specific to merge-based evolution.

5.1 Overlapping I/O-bound rollouts

Agent rollouts are dominated by time spent waiting on a model endpoint, so thread-level concurrency suffices to overlap them (Section 8.1 measures 5.8× at eight threads on real API calls). The synchronous data-parallel runtime shards tasks across N workers over a common snapshot and merges at a round barrier. The barrier, however, imposes a ceiling independent of N: with i.i.d. rollout latencies t_i,

$$T^{\text{sync}}_{\text{round}} = \mathbb{E}\big[\max_{i \le N} t_i\big] + T_{\text{gate}}, \tag{5}$$

and reasoning-model latencies are heavy-tailed — a short answer and a long deliberation are the same call — which maximizes the gap between $\mathbb{E}[\max_i t_i]$ and $\mathbb{E}[t]$. The second term is a serial region in which all workers idle while the gate evaluates.

5.2 Barrier-free execution under bounded staleness

The asynchronous runtime (Algorithm 1) removes the round structure: workers produce continuously, a dedicated merger consumes continuously, and per-rollout cost approaches $\mathbb{E}[t]$ — the no-barrier discipline of asynchronous RL systems [1, 7] transferred to self-evolution. Its cost is staleness, which Eq. (2) makes first-class and per-diff, bounded by two mechanisms. A *lag budget* ρ (`async_ratio`) limits drift at the source: a worker refreshes its snapshot once v_{head} exceeds its snapshot version by more than ρ, and backpressure forces a global resynchronization if evidence accumulates while nothing commits. A *staleness policy* disposes of stale diffs at the merger: FULL admits them unconditionally (sound only where diffs compose); GUARDED rebases within $\eta \le \eta_{\max}$ and discards beyond, in the spirit of bounded-staleness RL [7], paying for safety in discarded rollouts; REFLECTIVE always rebases and *re-verifies* — a stale diff is treated as unproven rather than invalid, and is discarded only if its claimed improvement fails to hold on the new head.

Algorithm 1 Barrier-free evolution (`async_evolve`); the synchronous runtime is the special case with a round barrier between the two loops. S is the strategy of Section 3.1. Backpressure has two parts: `buffer.put` blocks while the unmerged intake exceeds the lag budget, and a stall detector (evidence arriving while nothing commits) forces a global resynchronization. In the current implementation a single merger thread performs all commits, so the CAS never contends; multi-merger operation, where it would, is future work.

Worker i **(in parallel,** $i = 1..N$**):**
loop
  if $v_{\text{head}} - v_i > \rho$ **then** refresh snapshot $A_i \leftarrow$ ledger head; $v_i \leftarrow v_{\text{head}}$
  end if
  sample task x; rollout $\tau \leftarrow \pi_{A_i}(x)$; reward $r \leftarrow r(\tau, x)$
  $(d, e) \leftarrow S.\text{propose}(A_i, x, \tau, r)$; buffer.put$(d, e)$ ▷ blocks on backpressure
end loop
Merger (single thread):
loop
  $\mathcal{B} \leftarrow$ buffer.drain(); bucket $\mathcal{B}$ by artifact
  for each bucket with $|\mathcal{B}_a| \geq B$ or timed out **do**
    $\mathcal{B}_a \leftarrow \{\, d : \text{policy}(\eta(d), \eta_{\max}) \neq \textsc{discard} \,\}$, rebasing where directed ▷ Eq. (2)
    resolve conflicts (keep better of each contradicting pair); $u \leftarrow$ fuse(survivors)
    if survivors contradict on a key **and** reflective merge enabled **then** $u \leftarrow$ model-written union
    end if
    if audit passes **and** Eq. (3) holds on D_{val} **then** CAS-commit u to `dev`
    end if
  end for
  promote `dev` $\rightarrow$ `stable` after K regression-free rounds
end loop

Because ρ is denominated in artifact versions, its appropriate value scales with rollout cost; the runtime emits a warning when a run discards the majority of its evidence.

5.3 Reducing validation cost

The third source of speedup is unavailable to the first two mechanisms: the gate itself. Every acceptance decision costs one sweep over D_{val}, and on live models this dominates the merger's critical path: profiled on a live GEPA/HotpotQA run (N=4, `GLM-5.2`), the merger thread is busy 560.1 s of a 740 s process — 75.7% occupancy on a single thread. We report the occupancy and not a gate/non-gate split: in the released instrumentation the two timers wrap the same region, so their ratio is 1.0 by construction rather than by measurement, and a decomposition of the merger's time into evaluation, ledger I/O and commit needs instrumentation this paper does not have. A serial or selection-based loop with m proposals pays $m \cdot |D_{\text{val}}|$ gate evaluations. Merging changes the count: a round's k surviving diffs fuse into one candidate, so per-round gate cost falls from $k \cdot |D_{\text{val}}|$ to $|D_{\text{val}}|$; a diff discarded by the staleness filter never reaches the gate and costs zero evaluations; and in the asynchronous runtime, cards arriving during a merge enlarge the next batch rather than triggering additional merges. The effect is visible directly in the RQ1 experiment (Fig. 5): model-seconds per rollout fall across the serial, synchronous, and asynchronous arms at a pinned rollout budget. Two costs partially offset the saving and are stated for balance: the REFLECTIVE staleness policy's re-verification spends additional gate evaluations, and a model-written reflective merge spends one further model call per contested fusion. Residual gate cost parallelizes independently of the worker pool (`eval_concurrency`), saturating at $|D_{\text{val}}|$.

The reduction presupposes that proposals *can* fuse, which fails exactly when the artifact's key space is too coarse. An artifact held in a single key — e.g. a prompt as one instruction slot, GEPA's setting [4] — makes every pair of concurrent proposals a write–write conflict; a keyed union then degenerates to retaining the better single proposal, i.e. best-of-n selection at best-of-n validation cost. *Reflective merge* restores the reduction: a model is prompted to write the *union* of the contradicting proposals — one candidate preserving each proposal's intent — and the union passes through the same acceptance test as any candidate, so the model proposes and the gate decides. Section 8.2 isolates this on a one-key artifact and measures what it is worth: the keyed union fuses in 0 of 48 merges there and reflective merge in 42, for 47% fewer model calls at an identical rollout budget. It is the one place in this paper where a mechanism is measured with every alternative explanation removed by construction, because on a one-key artifact nothing else can fuse. Key granularity is necessary but not sufficient, and the missing condition is quantifiable. The engine proposes only from a rollout that *failed* — a solved task yields no diff — so if the incumbent fails a fraction f of tasks and N workers run

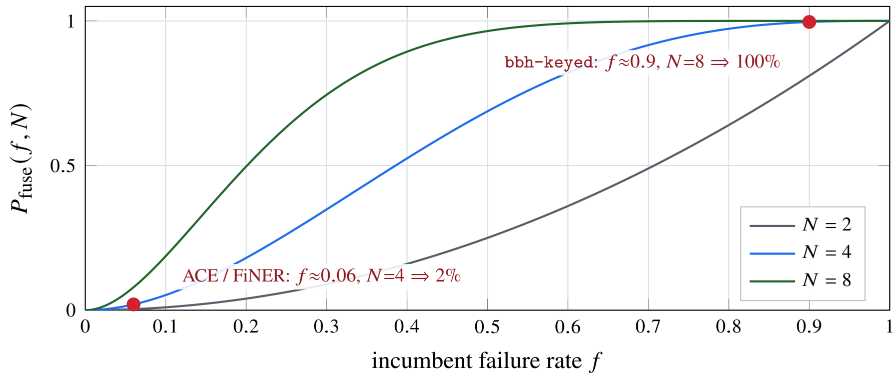

Figure 2: The fusion precondition of Eq. (6): the fraction of rounds in which at least two diffs exist, and therefore anything to fuse at all. Because the engine proposes only from *failed* rollouts, a near-saturated artifact starves the merge step regardless of how finely its keys are cut — the curve collapses at small f for every width. Red markers are the two workloads of Section 8.4: the one whose merge step never fired sits at 2%, and the one built to satisfy the condition sits at 100%. Widening N moves the curve left, which is how the equation bounds fusion width *from below*; the round count and the trust region bound it from above.

per round, the probability that a round produces at least two diffs, and therefore anything to fuse, is

$$P_{\text{fuse}}(f, N) \;=\; 1 - (1-f)^N - Nf(1-f)^{N-1}. \tag{6}$$

This is a *necessary* condition and an upper bound, not a prediction of the fusion rate, and the gap between the two is large enough to state plainly. Equation (6) counts rounds with two failed rollouts; between a failed rollout and a diff in the aggregator's bucket sit several further filters that it ignores — the proposer can return nothing, the strategy can fail to encode a proposal as a diff, the staleness filter can discard, and the aggregator buckets *per artifact*, so two failures touching different artifacts produce nothing to fuse. Measured against the runs of Section 8.4, where $f \approx 0.9$ puts the bound at essentially 100%: at N=8 the aggregator contested 10 of 24 rounds (42%) and at N=4, 21 of 144 (15%) — the bound overpredicts by 2.4× and 6.8×. So Eq. (6) is useful in one direction only: when it is *small*, merging cannot fire and no experiment on that workload can measure merging, which is exactly how it retrodicts our two nulls. When it is large, merging may still rarely fire, and only the `contested` counter settles it. A near-saturated artifact starves the merge step no matter how finely its keys are cut; a far-from-saturated one merely permits fusion. Section 8.4 measures both sides of it. It is the spatial counterpart of the REFLECTIVE staleness policy (carry intent onto a changed base; let measurement decide), and the `contested` counter reports whether fusion genuinely occurred, so the saving is never claimed where only selection took place.

6 Governance by Blast Radius

Artifacts are assigned to layers by a declared `blast_radius`: **L2** (skills, prompts) merges under the full asynchronous pipeline; **L1** (harnesses, verifiers) serializes in-flight changes and routes every merge through the oracle; **L0** (the oracle, audit budget, merge permissions, safety constraints) is frozen and read-only to the loop. The frozen layer enforces the second disanalogy of Table 1: without it, a self-referential loop can converge to a verifier that accepts its own output. The same layering protects executed agent code, which runs behind a frozen test suite the candidate cannot rewrite; pristine tests are overlaid after workspace materialization, so weakening them at run time is likewise ineffective.

7 Case Study: Porting Eighteen Self-Evolution Algorithms

Because every component sits behind a seam (Section 3.3), porting a published algorithm requires a small set of plug-ins rather than a fork. Two routes cover all eighteen ports.

Route 1: a strategy, plus an optimizer where needed. A *benchmark-faithful* port supplies a `strategy` encoding the paper's artifact as keyed diffs and, only when its search differs from the default pipeline, an `aggregator_factory`. The seven faithful ports illustrate the scale of the required code: ACE [39] is a playbook strategy whose curator *is* the default aggregator; GEPA [4] substitutes a per-instance Pareto optimizer; ADAS [9] and DGM [37] plug in keep-all archives with their own parent selection; EvoSkill a bounded top-K frontier; SkillOpt a strict edit gate;[1] OpenEvolve [26]

[1]EvoSkill and SkillOpt are ports of released systems whose upstream papers and per-port departures are identified in the repository's port-fidelity documentation; we do not cite them here to avoid attributing a reconstruction to the wrong reference.

MAP-Elites islands [20]. Parent-selection and acceptance rules are extracted as named policy classes, so subsequent ports reuse them.

Route 2: a declarative policy bundle. The eleven microports and analogues (PromptBreeder, AFlow, Self-Refine, Reflexion, SICA, Gödel Agent, Voyager, SkillWeaver, Absolute Zero, R-Zero, Agent0) define no new classes: each is a `MethodPolicy` — selection rule, memory, acceptance shape, genome — over one shared runner, inheriting a common budget contract, dataset layer, and command line. A shared `--serial` flag reproduces the upstream algorithm's own semantics (one worker, serial gate) as the control that any parallelization claim requires, and a shared rollout budget pins compared arms to equal work.

Fidelity is declared per port — *benchmark-faithful*, *mechanism microport*, *self-edit analogue*, *environment analogue*, or *inference analogue* — follows the released code where it diverges from the paper, and documents infrastructure boundaries explicitly; analogues are not to be cited as benchmark reproductions. All eighteen ports run under serial, synchronous, and barrier-free scheduling without modification, which is what makes the scheduler a controlled variable rather than a property of each paper's own loop.

8 Evaluation

Setup. All results are obtained from live models on real datasets: `deepseek-v4-flash` and `GLM-5.2` (reasoning models) via an OpenAI-compatible endpoint, on each algorithm's own benchmark. Every number is produced by a named script in the repository with its exact reproduction command; absolute times depend on host and endpoint, so ratios are the reproducible quantity. We address four questions: **RQ1** — how much effective concurrency do the three mechanisms of Section 5 deliver against a faithful serial control? **RQ2** — what does reflective merge buy, on an artifact where nothing else can fuse? **RQ3** — is learning behavior preserved under the most aggressive schedule, and how optimistic is the gate that decides it? **RQ4** — does merging accelerate without a quality penalty relative to best-of-N selection at equal budget?

Statistical conventions. Spreads across seeds are sample standard deviations. Sign tests are exact and two-sided with ties dropped, and the unit of analysis is stated with each one. No correction for multiple comparisons is applied anywhere; the paper makes on the order of a hundred pairwise comparisons, so a single nominal p near 0.05 should be read as descriptive. Where a design cannot reach significance — which is the case for every RQ4 quality comparison — we report paired confidence intervals and the minimum detectable effect instead of a p-value.

Table 2 states every configuration in full, including the two runs that produced null results and are reported as such. Three conventions hold throughout and are worth stating once, because each of them costs us something. The rollout budget is *pinned on every arm* of every comparison, so a wide arm cannot buy its speedup with extra work — the failure mode that makes an unbudgeted N=8 arm look eight times better. The serial control runs its gate at `eval_concurrency=1`, reproducing the published loop rather than a partly-parallel version of it; this makes the control slower and every speedup beside it larger, which is why the value is recorded rather than assumed. And the held-out *test* split is disjoint from the gate's D_{val} in every experiment, so no quality figure below was ever seen by an acceptance decision.

8.1 RQ1: Effective concurrency against a faithful serial control

We first verify the premise of Section 5.1: eight threads over one pool reduce a real `GLM-5.2` API workload from 48.6 s to 8.3 s (5.8× in this run; 5.8–6.5× across three repeats, sub-linear under heavy-tailed latency), while the same threads on pure-Python CPU work yield 1.1× — rollout overlap is real, and the GIL does not limit I/O-bound rollouts.

The main experiment ports GEPA to its own benchmark (HotpotQA [4, 33]) with 16 rollouts pinned on every arm, three seeds, and a control that reproduces the upstream loop exactly: one worker *and* a serial gate. That control lands at 0.99, 1.00, and 1.00× overlap on the three seeds (Fig. 4), which is worth stating on its own: the property that makes it a faithful control is stable, not a fortunate single run. Against it, synchronous parallelism spans 3.18–3.96× effective concurrency (median 3.91) and barrier removal spans 3.30–6.44× (median 6.28); median wall-clock falls 790 → 152 → 116 s (Fig. 3) — a 6.8× reduction in the time to spend the same rollout budget, which is the framework's core claim in its most direct form. Figure 5 shows the validation-cost mechanism of Section 5.3 in the same runs, and it replicates across seeds: model-seconds per rollout fall from a median 49.3 (serial) to 37.6 (synchronous) to 36.7 (asynchronous) at the pinned budget — the parallel arms not only finish sooner, they evaluate less per unit of work. Independent endpoint probes sustain 7× (matched width and request length) to 10× (wider, shorter requests) concurrency, so no arm is provider-limited.

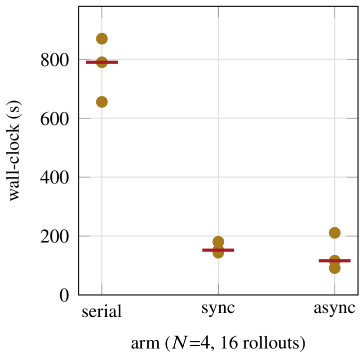

Figure 3: RQ1, the quantity the framework exists to reduce: wall-clock to spend a pinned rollout budget. GEPA on HotpotQA, `GLM-5.2`, 16 rollouts on every arm, three seeds, medians marked. 790 → 152 → 116 s: parallelism takes the first 5.2×, barrier removal a further 1.31×, 6.8× end to end.

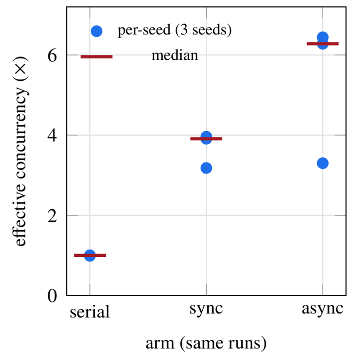

Figure 4: Where that time goes: effective concurrency (model-seconds ÷ wall-clock). The serial control reproduces the published loop (one worker, serial gate) and lands at 0.99–1.00× on every seed — a stable control property, not a single-run coincidence. Synchronous parallelism spans 3.18–3.96×; barrier removal spans 3.30–6.44×, the widest arm, and reaches it while overrunning the pinned budget (19 rollouts against 16).

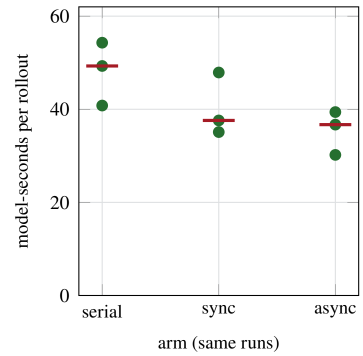

Figure 5: Validation-cost reduction in the same runs (Section 5.3): model-seconds spent per rollout at a pinned rollout budget, per seed, with medians marked. The parallel arms evaluate less per unit of work — median 49.3 → 37.6 → 36.7 — but the asynchronous figure is normalized by *measured* rollouts, and that arm ran 19 against the pinned 16. Charged against the budget every arm was given, it is $36.7 \times 19/16 = 43.6$, above the synchronous arm's 37.6: the serial-to-synchronous step survives the correction and the synchronous-to-asynchronous step does not. We do *not* attribute this to fusion: RQ1's artifact is GEPA's single-key instruction slot, on which a keyed union is never contested, so the collapse of Section 5.3 cannot be what fired here. The candidates are the admission cap (one merged candidate per round, so the Pareto pool grows by one instead of N) and staleness discards, which reach the gate at zero cost. Separating them needs an arm without the cap, which we have not run.

Three qualifications bound these numbers. First, *the asynchronous arm is not paying the same bill*: lacking a round boundary it ran 19 rollouts against the pinned 16 on every seed, so part of its concurrency is work the other arms did not perform. Second, *it is also the least stable arm* — a 3.30–6.44× spread against synchronous parallelism's 3.18–3.96×. Third, and most important, *none of this is a quality claim*: median held-out test score is 0.55 (serial), 0.60 (synchronous), 0.50 (asynchronous) across the three seeds, on a 20-item split whose resolution is 0.05, so the arms are indistinguishable in quality and the fastest arm is not the best one. One synchronous cell (seed 0) also ended early on an endpoint error at 12 of 16 rollouts and is included as measured.

These figures supersede a single-run measurement of 1.85 × /3.22× on a different model reported in an earlier draft. Two differences account for the gap and neither is a correction of the other: this run uses `GLM-5.2` rather than `deepseek-v4-flash`, and its parallel arms run the gate at `eval_concurrency=16`. The second is the more instructive: with the gate itself parallelized, the round barrier costs much less, which both raises the synchronous arm and shrinks what barrier removal has left to recover — the regime dependence that decides when removing the barrier pays at all. An earlier draft additionally reported a width-8 variant with a 28% model-call saving; it is withdrawn because its serial arm ran with a concurrent gate, violating the control definition above. The shipped cell for that run records 3849.7 model-seconds inside a 2148.4 s wall-clock, i.e. 1.79×; an earlier draft quoted 1.31× for the same cell and that figure was wrong.

8.2 RQ2: What reflective merge buys, isolated

The keyed union of Section 4 fuses diffs that touch different keys. When an artifact is held in *one* key, every pair of concurrent proposals is a write–write conflict, the union cannot be formed, and merge-of-N collapses into best-of-

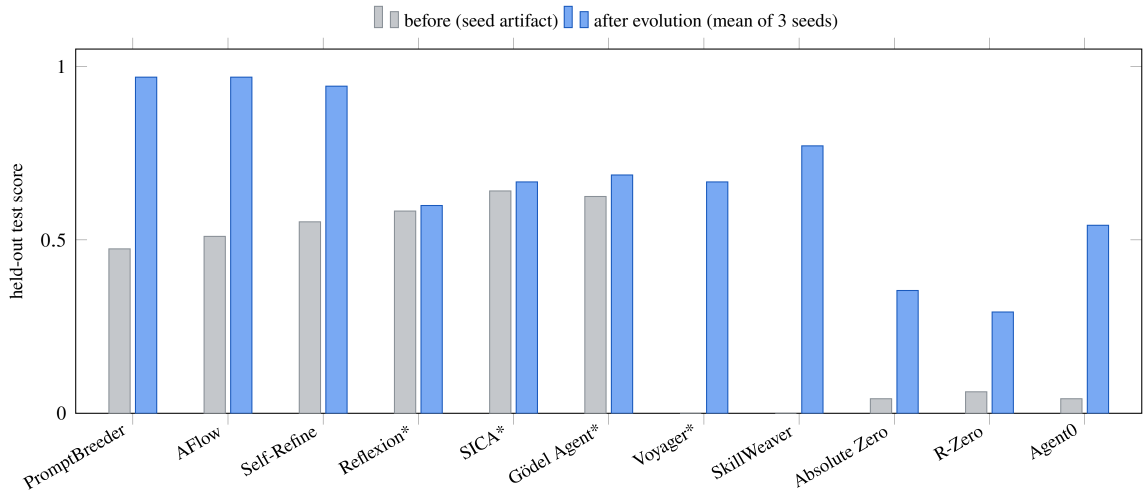

Figure 6: Held-out test score before and after evolution for the eleven microports and analogues, each run against `deepseek-v4-flash` under the *most aggressive* configuration — barrier-free async pipeline, 8 workers, Full staleness, 80 rollouts per seed, 3 seeds (465–2,069 model calls per seed). "After" bars are three-seed means; asterisked methods improved on two of three seeds (the rest on all three), and Reflexion's gain is within its domain's ±0.02 noise floor. Between 3 and 10 of the 240 proposals per method (80 × 3 seeds) commit — the gate admits only strict held-out improvements, so sparse acceptance is the shape of the mechanism, not a low yield. Bars are comparable only within a domain; per-seed spreads are tabulated in the repository documentation.

N selection — while still paying best-of-N validation, since each surviving proposal is scored separately. *Reflective merge* is the mechanism that is supposed to close that gap: a model writes one value preserving what each proposal contributed, and the union goes through the same acceptance test as any other candidate. Everything said about it so far in this paper has been a design argument. This experiment measures it.

The workload is chosen so the mechanism is the *only* thing that can fuse: BBH `dyck_languages` [29] behind an `InstructionSlot`, a one-key artifact on which a keyed union is never available. Two configurations differ in a single flag — 48 rollouts pinned, N=4, four seeds, the same 60/30/30 split, the same gate. OFF runs the keyed union with the fusion tournament enabled so the degeneracy is visible in the counters; ON runs reflective merge.

The degeneracy is exact, not approximate. Across four seeds the OFF configuration held 48 merges and contested *none* — every one recorded as single-candidate, none as contradicting. This is the strongest form the prediction could take: on a one-key artifact the keyed union does not fuse rarely, it never fuses. Under ON the same 48 merges produced 42 fused unions.

The mechanism halves validation cost. At a pinned rollout budget — identical work, identical gate — reflective merge spends 528 model calls against 1001, a 47% reduction. The effect is in the same direction on every seed (per-seed ratios 0.470, 0.509, 0.553, 0.597; paired difference −472 calls, 95% CI [−634, −311]), and its shape matches the mechanism rather than merely its sign: ON's cost is nearly constant across seeds (523–534) because one union is scored per round whatever the round contained, while OFF ranges 888–1113 because it pays per surviving proposal. This is Section 5.3's third source of speedup measured in isolation, on the one artifact shape where no other mechanism could have produced it.

Quality: no difference detectable at this resolution, and the resolution is poor. Mean held-out test is 0.250 (OFF) against 0.234 (ON); the paired difference is −0.017 with a 95% interval of [−0.256, +0.223]. Four seeds on a 30-task split detect only effects larger than about 0.36 — wider than the arm means themselves — so this experiment establishes the cost result and is simply not powered for the quality one. We report the interval rather than a p-value for that reason, and we do not claim equivalence.

8.3 RQ3: Quality preservation under the aggressive schedule

Throughput is meaningful only if learning is preserved, so quality is measured under the most aggressive configuration (barrier-free, 8 workers, FULL staleness). Figure 6 reports the eleven declarative ports at three seeds each. Read precisely, and more conservatively than an earlier draft did. Ten of eleven raise their three-seed mean test score, but a per-port paired t-test on three seeds ($t_{.975,2}$ = 4.303) separates only six of them from zero: Reflexion, SICA, Gödel

Agent, R-Zero and Voyager do not, and four of those five are the methods that improved on two of three seeds rather than all three. The ±0.02 *noise floor* an earlier draft applied uniformly is also not defensible across these splits, which range from 64 items to 8: on SkillWeaver's eight-task split one task is 0.125, and Voyager's test metric takes only the values 0 and 1, so a ±0.02 floor is arithmetically impossible there. Bars are comparable only within a domain and the per-port n differs by a factor of eight. Gains are bounded by each benchmark's headroom — the three GSM8K ports converge to 0.94–0.97 and differ in cost rather than quality. What this establishes is that the most aggressive schedule the framework offers — no barrier, eight workers, staleness admitted unconditionally — does not stop published algorithms from learning on their own benchmarks; a paired serial-schedule arm at equal budget would additionally price asynchrony *relative to serial*, and is left to future work. Among the benchmark-faithful ports, three representative single-seed results establish end-to-end operation (not effect sizes): ACE improves FiNER-139 [18] validation 0.719 → 0.766 over 120 rollouts while curating a ten-bullet playbook; DGM's best archived child repairs held-out vendored bugs at 0.906 versus its seed agent's 0.844 under a frozen `pytest` oracle (a compact SWE-bench-style setting [13]); SkillOpt improves a hard SearchQA [5] subset 0.053 → 0.211 with 3 of 43 proposed edits accepted. One port produced no quality number at all: ADAS's benchmarks were either saturated (MGSM) or cost-prohibitive per call (GPQA), and we state this rather than average over it. The highest-level interface (`evolve_skill`) on 40 HotpotQA rows raises held-out exact match from 0.167 to 0.583 over 338 model calls, committing one proposal.

Is the gate that decides this optimistic? It reuses one fixed D_{val} across every decision, so an accepted diff is a selected extreme statistic on that split (Section 4). The 33 runs above measure the resulting bias directly, since each records both its D_{val} trajectory and its disjoint test score: mean val gain +0.395 against mean test gain +0.357, a mean gap of −0.038 (median −0.031, s.d. 0.075), with test gain short of val gain in 20 of 33 runs and above it in 7 — a two-sided sign test on the 27 non-tied runs gives $p = 0.019$. That test treats the 33 runs as independent, which they are not: they are 11 ports × 3 seeds, and seeds within a port share a benchmark, a D_{val}, and a reward granularity. Taking the port as the unit instead gives 8 negative port means against 2 positive and 1 tied, $p = 0.109$ — not separable. The run-level result is also fragile: dropping any one of five of the eleven ports moves p above 0.05, and dropping SkillWeaver alone — whose test split has eight items, so one task is 0.125 — moves the mean gap from −0.038 to −0.026. What we claim is therefore the direction and the magnitude, not significance: the optimism is consistently negative, about a tenth of the mean gain, concentrated in the coarsest-reward ports, and too small to change the sign of any result here, where every quality figure is reported on test.

8.4 RQ4: Merge-of-N versus best-of-N selection at equal budget

Throughput cannot distinguish merging from sampling-and-selecting, since N independent forks occupy N workers equally well; the discriminating quantity is held-out quality at an equal rollout budget. The largest such run to date — HotpotQA on `GLM-5.2`, three arms × three seeds at nine rollouts per seed, budget pinned in rollouts, reflective merge enabled so that contradicting proposals survive to fusion; 1,298 model calls (29 failing at the endpoint) and 11.3 hours of model time — does not separate the arms: merge-of-3 attains the highest median test score together with the widest spread, the intervals overlap, and the framework's own separation test declines to declare a winner. The diagnosis is the useful part, and it is what produced Eq. (6): at this budget the fused union was constructed twice in the entire experiment — most merges saw a single surviving diff — so the experiment does not measure a merging deficit; it measures a regime in which merging rarely fired. It also confirms the key-granularity condition of Section 5.3 from the opposite direction: without reflective merge, the merge arm on this single-key artifact would have reduced to best-of-N selection. We therefore tested that diagnosis on a *multi-key* artifact: ACE's playbook on FiNER-139, keyed per bullet, so two workers editing different bullets compose by construction. Fusion still did not fire — over a 24-rollout, four-worker run the aggregator held **2 tournaments and contested none**, one with a single candidate and none with contradicting candidates. The cause was not key granularity but Eq. (6): that artifact already scored 0.938 on the gate's split, so $f \approx 0.06$, and at N=4 the predicted fraction of rounds with anything to fuse is 2%. Two uncontested tournaments in six rounds is what a 2% rate looks like.

Equation (6) also prescribes the fix, so we built the workload it calls for. BBH [29] supplies tasks a strong model still fails, and `KeyedRules` supplies an artifact keyed by the category a failure diagnosis falls into (output format, method, verification, common errors). Crossing them needed no engine change — only a new `Workload`, because dataset and strategy are independent seams (Section 3.3). On `GLM-5.2` the seed artifact scores 0.100 on test, so $f \approx 0.9$, and at N=8 Eq. (6) predicts that essentially every round should have something to fuse.

It does. Over a 32-rollout run the aggregator held **3 contested tournaments** — the first in this project in which fusion actually ran — while the artifact improved from 0.100 to 0.400 on test, so the workload supplies both the failure rate and the headroom the question needs. The fused candidate **won 1 of the 3**, at a mean gain of −0.115 with two losses (worst −0.250); two of the three fell below the artifact they started from. On this workload, fusing a round's diffs

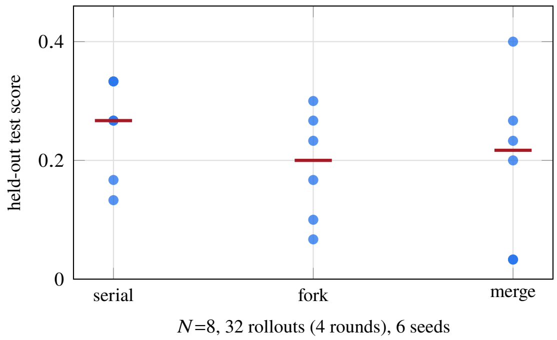

Figure 7: Fusion width 8. Per-seed test scores with medians marked. The three arms overlap; the merge arm has the *widest* spread (s.d. 0.142 against 0.084 and 0.093).

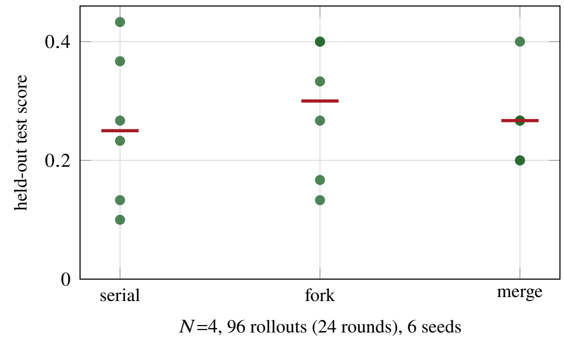

Figure 8: Fusion width 4, matched to the trust region, at a 3× budget, six seeds. The arms still overlap, but the merge arm is the *tightest* (s.d. 0.073 against 0.129 and 0.115) — the ordering that width 8 reverses.

was usually *worse* than keeping the better single one — and the acceptance gate refused exactly those, which is the safeguard of Section 4 behaving as designed rather than an argument against it.

With fusion firing on demand, the comparison the question has been waiting for becomes possible. We ran it at three seeds, and then — because the three-seed reading was favorable to merging — at three more. The second half did not replicate the first, and the combined result is the one we report: six seeds × three arms on `bbh-keyed`, 32 rollouts pinned on every arm, N=8, `GLM-5.2` (16,124 model calls, 28.3 hours of model time).

Merging is the cheapest arm at equal quality. The framework's premise is throughput — accepted improvement per unit wall-clock — so the claim merging has to meet is non-inferiority in quality at lower cost. The six seeds deliver the cost half cleanly and the quality half only weakly. Quality: the paired merge−fork difference is +0.005 with a 95% interval of $[-0.186, +0.196]$ at N=8 and −0.017 with $[-0.133, +0.100]$ at N=4. Those intervals are wider than the arm means themselves (0.19–0.28), so what the design can detect is a penalty of roughly 0.16–0.26 test-score points and nothing smaller. We report the intervals rather than a p-value because at six seeds with ties the sign test cannot reach $\alpha = 0.05$ at all — its smallest attainable p is 0.0625 at N=8 and 0.125 at N=4 — so a non-significant result there carries no information. Cost, by contrast, is measured cleanly: merge is the cheapest arm in model calls — 11% fewer than serial and 22–32% fewer than fork. Against fork this is the validation-cost mechanism of Section 5.3 showing up as money (Fig. 9): N independent forks must each be scored where a round's survivors are scored once. Against *serial* it cannot be, and we say so rather than let the figure imply otherwise: a serial lineage produces one proposal per step, so $k = 1$ and collapsing k tests into one predicts no saving at all. The 11% we measure there must come from something else — fewer proposals, because merge workers share what the others have already learned and more of their rollouts solve outright; or diffs discarded by the staleness filter before they reach the gate. Separating those needs a decomposition of calls into rollout, proposal and gate-evaluation counts, which the engine's counters permit and we have not reported.

We report no wall-clock advantage over fork, and an earlier draft's claim of one was wrong. The fork arm's runs were executed sequentially (`--fork-concurrency 1`), so the wall-clock we compared against was the *sum* of N forks rather than what N concurrent forks would take. On the same runs the longest single fork is 320–376 s against merge's 1,127–1,176 s: given N workers, fork-of-N finishes a pinned rollout budget *faster* than merge-of-N, because each fork is a short serial lineage with no round barrier and no shared gate. That is the honest comparison and it cuts against us on this axis. Merging's cost advantage is in model calls, where the gate is amortized; wall-clock at equal worker count belongs to selection.

Two bounds travel with the quality column. Seeds 0–2 alone put merge's median at 0.267 against fork's 0.100 and seeds 3–5 reversed it; we report the pooled six rather than the favorable half. The decision to run the second three was made *after* seeing the first three and because they were favourable, which is data-dependent stopping: the six-seed figures are therefore descriptive, and we do not attach inferential weight to them beyond the intervals above. And six seeds on a 30-task split detect only large effects, so what is established is the absence of a large quality penalty.

Fusion width is a design parameter, and Eq. (6) does not fix it alone. The precondition is satisfied almost equally by N=4 (99.6% of rounds have something to fuse) and N=8 (100%), but the two differ in what they do to the rest of the loop: at a fixed rollout budget N=8 halves the number of rounds, and therefore the opportunities to accumulate, while making each fused union an eight-way jump that the trust region of Section 4 exists to prevent. Width should therefore be set at the smallest N that clears Eq. (6), not the largest available — a concrete design rule the equation yields once

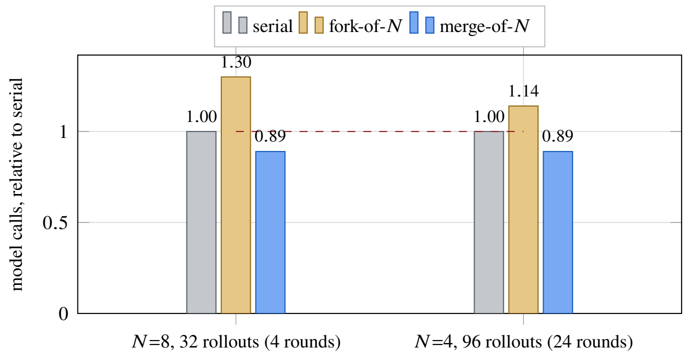

Figure 9: What the arms of Table 4 cost, at equal held-out quality and an equal rollout budget, normalized to the serial control in each configuration (absolute counts: 772/1004/685 and 2070/2363/1832). Merging is the cheapest arm in both configurations and by the same margin — 11% below serial and 22–32% below fork-of-N — because a round's surviving diffs are validated once rather than once each. Selection has no such saving available: N independent forks must each be scored, which is why the fork arm sits *above* the serial control despite doing the same number of rollouts.

the round count is priced in. Re-running at N=4 and a 3× budget (24 rounds), also at six seeds, moves the mechanism in the predicted direction: the mean per-seed fusion gain rises from −0.135 to −0.045, and the merge arm's seed-to-seed standard deviation falls from 0.142 — the *worst* of the three arms — to 0.073, the *lowest*, against serial's 0.129 and fork's 0.115. None of these variance ratios is separable at six seeds (every F is far inside the two-sided critical value at $5, 5$ degrees of freedom), and the ordering reverses between the two widths, so we report it as an ordering with a mechanism attached and not as an effect. Quality remains indistinguishable within intervals as wide as those above, and merge remains the cheapest arm (1832 calls against 2070 and 2363).

That variance ordering is the observation we would most like to see tested properly, and six seeds cannot test it. It has a mechanism to its name — merging averages several workers' evidence into one artifact, where selection keeps whichever single lineage the gate happened to prefer — but the evidence has three problems we state rather than argue past: no variance ratio is significant at n=6, the ordering *reverses* between the two widths, and the ratio halved when we went from three seeds to six. A quantity that flips across configurations and shrinks as the sample grows is what noise looks like. We record it as a hypothesis with a mechanism, not as a finding.

The fusion counters, across both configurations. At N=8 the aggregator held 10 contested tournaments over six seeds and the fused candidate won 2 (20%); at N=4 it held 21 and won 4 (19%), at a mean per-seed gain of −0.045 against width 8's −0.135. Fusion loses more contests than it wins in both — and this is why the design works: the acceptance test refuses exactly those losses, so carrying a usually-worse union costs one gate sweep and nothing else, while the wins are kept. Non-inferior quality at a 20% fusion win rate is the gate doing its job, not a result in spite of it. The negative control holds throughout: across eighteen control runs the serial and fork arms held 823 tournaments and contested *none*, as their construction requires.

The summary is therefore narrower than an earlier draft of this paper claimed: **on this workload merging costs** 11–32% **fewer model calls than the alternatives at a pinned rollout budget, and we cannot detect a quality difference at the resolution six seeds afford.** The cost result is the measured one; the quality result is an interval too wide to exclude penalties smaller than about 0.16. Merging is not shown to improve quality, and it does not win on wall-clock.

9 Limitations

AgentDescent is a research reference implementation, and four boundaries delimit what its evaluation establishes. (i) *Effect sizes*. The quality results establish direction and rule out large penalties rather than measuring magnitudes: six seeds on a 30-task split in RQ4, four seeds for the reflective-merge ablation, three for the RQ1 ladder and the eleven-port study, one seed for each benchmark-faithful port. Merging is shown non-inferior and cheaper, not superior, and the variance ordering of Section 8.4 is reported as a direction rather than a ratio. (ii) *Scale*. The system is a single Python process — one merger thread (measured at 75.7% occupancy at N=4), thread workers, a local git ledger. Multi-machine operation, a contended CAS path, and a worker sweep beyond N=8 are outside this paper. We also cannot yet say *what* the busy merger is busy with: the `merge_seconds` and `merge_gate_seconds` timers wrap the same region in the released code, so the profile bounds the merger's occupancy but does not attribute it. Locating the saturation point needs both that decomposition and the sweep. (iii) *Holdout reuse*. The gate reuses one fixed D_{val} across all acceptance decisions (Section 4); test splits are disjoint and the resulting optimism is measured at 0.038 (Section 8.3), but no reusable-holdout or D_{val}-rotation mechanism is implemented. (iv) *Transfer*. A multi-algorithm scheduler matrix reported in an earlier draft is withdrawn from this one: it was measured at N=2 on the implementation preceding the ports' declarative restructuring, its raw per-run data was not retained, and at two workers the round barrier has almost nothing to hide, so it could not distinguish a property of the ports from a property of the width. Separately, difficulty parameters calibrated on one model did not transfer under a model swap, whereas benchmark-intrinsic difficulty did. Every number reported here is generated by a named script, and measurements that proved irreproducible or favorably defined — including, in an earlier draft, a width-8 experiment withdrawn for a flawed control — were re-measured and corrected in place. We regard that discipline as part of the contribution: the evaluation of a self-improving system should satisfy the standard its own acceptance gate imposes on diffs.

10 Conclusion

The distributed-training stack admits a coherent counterpart in agent self-evolution once its necessary disanalogy — diffs do not add — is taken seriously and merging is constructed as a discrete-space optimizer. Efficiency decomposes into three sources, and the third is the one specific to merging: rollout overlap and barrier removal take wall-clock from 790 to 116 s against a faithful serial control, while reduced validation cost — k acceptance tests collapsed into one — is the saving selection cannot have. That third source is where this paper spends its evidence, because it is the one with a precondition that can fail silently. Eighteen published algorithms adapt as plug-ins rather than forks — eleven of them measured here, the rest ported but not used as evidence — and all run under the three schedulers unmodified — which is what makes the scheduler a controlled variable instead of a property of each paper's own loop. Merging has two preconditions and both can fail quietly. It needs two diffs to exist, which Eq. (6) states in closed form and which a near-saturated artifact violates however finely its keys are cut; and it needs them to be fusable, which a one-key artifact violates absolutely — measured here as 0 fused unions in 48 merges. *Reflective merge* is what removes the second failure, and isolating it gives the paper its clearest result: 42 of 48 merges fused, and 47% fewer model calls at a pinned rollout budget, with a per-seed cost profile that is nearly flat because one union is scored per round however many proposals the round produced. Quality is not the claim; four seeds on a 30-task split cannot support one, and we report the interval rather than assert equivalence. What the result does support is a way of accounting we would like to see adopted: a self-evolution loop should be judged on accepted improvement per unit of *validation*, not per proposal, because validation is what it actually pays for.

Acknowledgments

OpenEvolve and the eleven microports and analogues were contributed by `cyanneko`.

Reproducibility

All code is available at https://github.com/Birfy/agentdescent (MIT license; `pip install agentdescent`; the core engine has no required dependencies). Generating scripts, per result: thread overlap — `examples/efficiency.py --only gil`; RQ1 and the live merger profile of Section 5.3 — `bench/matrix_run.py --rows gepa`; RQ2 — `bench/baselines_run.py --dataset bbh --fetch 120`, run twice, once with `--fusion-tournament` and once with `--reflective-merge`; RQ3 — each port's own command line over `examples/_method_runner.py`, the entry point that accepts the `--staleness full` configuration used, with the resolved configuration printed as one JSON line per run; the `evolve_skill` case — `examples/skill_evolution.py`; RQ4 — `bench/baselines_run.py --dataset bbh-keyed --fusion-tournament`, whose workload (BBH crossed with `KeyedRules`), width and budget sweeps, and fusion counters were added for this paper and ship with it. Every port supports `--dry-run` (no API key or model call) and carries an offline test suite; datasets are retrieved from canonical sources by a dependency-free data layer.

References

[1] ROLL Flash: Accelerating RLVR and agentic training with asynchrony. arXiv preprint, 2025. Preprint; identifier omitted pending verification.

[2] Agent0: Unleashing self-evolving agents from zero data via tool-integrated reasoning. arXiv preprint, 2025. Preprint; identifier omitted pending verification.

[3] Self-evolving agent skills via co-evolutionary verification (CoEvoSkills). arXiv:2604.01687, 2026. Concurrent work; official code unreleased at the time of writing.

[4] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, et al. GEPA: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2025.

[5] Matthew Dunn, Levent Sagun, Mike Higgins, V Ugur Guney, Volkan Cirik, and Kyunghyun Cho. SearchQA: A new Q&A dataset augmented with context from a search engine. *arXiv preprint arXiv:1704.05179*, 2017.

[6] Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution. *arXiv preprint arXiv:2309.16797*, 2023.

[7] Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, et al. AReaL: A large-scale asynchronous reinforcement learning system for language reasoning. *arXiv preprint arXiv:2505.24298*, 2025.

[8] Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. Connecting large language models with evolutionary algorithms yields powerful prompt optimizers. In *International Conference on Learning Representations*, 2024.

[9] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024.

[10] Chengsong Huang, Wenhao Yu, Xiaoyang Wang, Hongming Zhang, Zongxia Li, Ruosen Li, Jiaxin Huang, Haitao Mi, and Dong Yu. R-Zero: Self-evolving reasoning LLM from zero data. *arXiv preprint arXiv:2508.05004*, 2025.

[11] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. GPipe: Efficient training of giant neural networks using pipeline parallelism. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[12] Max Jaderberg, Valentin Dalibard, Simon Osindero, Wojciech M Czarnecki, Jeff Donahue, Ali Razavi, Oriol Vinyals, Tim Green, Iain Dunning, Karen Simonyan, Chrisantha Fernando, and Koray Kavukcuoglu. Population based training of neural networks. *arXiv preprint arXiv:1711.09846*, 2017.

[13] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2023.

[14] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations*, 2024.

[15] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

[16] Joel Lehman, Jonathan Gordon, Shawn Jain, Kamal Ndousse, Cathy Yeh, and Kenneth O Stanley. Evolution through large models. In *Handbook of Evolutionary Machine Learning*, pages 331–366. Springer, 2023.

[17] Mu Li, David G Andersen, Jun Woo Park, Alexander J Smola, Amr Ahmed, Vanja Josifovski, James Long, Eugene J Shekita, and Bor-Yiing Su. Scaling distributed machine learning with the parameter server. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*, pages 583–598, 2014.

[18] Lefteris Loukas, Manos Fergadiotis, Ilias Chalkidis, Eirini Spyropoulou, Prodromos Malakasiotis, Ion Androutsopoulos, and Georgios Paliouras. FiNER: Financial numeric entity recognition for XBRL tagging. In *Proceedings of ACL*, 2022.

[19] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[20] Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*, 2015.

[21] Feng Niu, Benjamin Recht, Christopher Ré, and Stephen J Wright. HOGWILD!: A lock-free approach to parallelizing stochastic gradient descent. In *Advances in Neural Information Processing Systems*, volume 24, 2011.

[22] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.

[23] Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A self-improving coding agent. *arXiv preprint arXiv:2504.15228*, 2025.

[24] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco J R Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.

[25] Jürgen Schmidhuber. Gödel machines: Self-referential universal problem solvers making provably optimal self-improvements. *arXiv preprint cs/0309048*, 2003.

[26] Asankhaya Sharma. OpenEvolve: an open-source evolutionary coding agent. https://github.com/codelion/openevolve, 2025.

[27] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[28] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

[29] Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V Le, Ed H Chi, Denny Zhou, and Jason Wei. Challenging BIG-Bench tasks and whether chain-of-thought can solve them. In *Findings of the Association for Computational Linguistics: ACL 2023*, 2023.

[30] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

[31] Mitchell Wortsman, Gabriel Ilharco, Samir Ya Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, et al. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on Machine Learning*, pages 23965–23998, 2022.

[32] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *International Conference on Learning Representations*, 2024.

[33] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of EMNLP*, 2018.

[34] Xunjian Yin, Xinyi Wang, Liangming Pan, Xiaojun Wan, and William Yang Wang. Gödel agent: A self-referential agent framework for recursive self-improvement. *arXiv preprint arXiv:2410.04444*, 2024.

[35] Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning. *Advances in Neural Information Processing Systems*, 33:5824–5836, 2020.

[36] Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. TextGrad: Automatic “differentiation” via text. *arXiv preprint arXiv:2406.07496*, 2024.

[37] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin gödel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*, 2025.

[38] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, et al. AFlow: Automating agentic workflow generation. *arXiv preprint arXiv:2410.10762*, 2024.

[39] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, et al. Agentic context engineering: Evolving contexts for self-improving language models. *arXiv preprint arXiv:2510.04618*, 2025.

[40] Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, and Gao Huang. Absolute zero: Reinforced self-play reasoning with zero data. *arXiv preprint arXiv:2505.03335*, 2025.

[41] Boyuan Zheng, Michael Y Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, et al. SkillWeaver: Web agents can self-improve by discovering and honing skills. *arXiv preprint arXiv:2504.07079*, 2025.

[42] Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. Large language models are human-level prompt engineers. In *International Conference on Learning Representations*, 2023.

Table 2: Every configuration behind Section 8. Budget is pinned in rollouts on all arms of a comparison unless noted. Flags not listed take each script's defaults, which every run prints in its header and records in its result JSON; Section 10 names the script per experiment. Model names are the endpoint's identifiers.

Experiment	Model	Workload	Arms, width	Budget, seeds	Gate, staleness, notes
Thread overlap (Section 8.1)	`GLM-5.2`	8 concurrent API calls of matched request length	8 threads vs. 1	3 repeats	Pure-Python CPU work run through the same harness as the GIL control
RQ1 (Section 8.1)	`GLM-5.2`	HotpotQA via the GEPA port; `--fetch 80`, `--val-cap 8`	serial / sync / async, N=4	16 rollouts per arm; seeds 0–2	Gate at `eval_concurrency=16` on the parallel arms, =1 on the serial control; `--async-ratio 3`; `--rounds 9999` so the budget binds, not the port's own iteration default; `--reflective-merge` on *every* arm, so it is not a difference between them; reasoning tokens disabled
RQ3 (Section 8.3)	`deepseek-v4-flash`	each port's own benchmark, 11 ports	barrier-free only, N=8	80 rollouts per seed; seeds 0–2	FULL staleness, the most permissive policy — stale diffs admitted unconditionally; 465–2,069 model calls per seed
RQ4, null 1 (Section 8.4)	`GLM-5.2`	HotpotQA; artifact is a single instruction slot	serial / fork / merge, N=3	9 rollouts per seed; seeds 0–2	Reflective merge on. 1,298 calls, 11.3 h model time. Reported as a null: the fused union was built twice in the whole experiment
RQ4, null 2 (Section 8.4)	`GLM-5.2`	FiNER-139 via the ACE playbook, keyed per bullet; `--pool 2000`, `--val-cap 32`	serial / fork / merge, N=4	24 rollouts; seed 0	Reflective merge and fusion tournament on. Reported as a null, and Eq. (6) explains it: the seed artifact already scored 0.938 on the gate's split, so $f \approx 0.06$
RQ2 (Section 8.2)	`GLM-5.2`	BBH `dyck_languages` behind an `InstructionSlot` — *one* key; `--fetch 120`, 60 train / 30 val / 30 test	merge only, N=4	48 rollouts per arm; seeds 0–3	Two configurations differing in one flag: OFF is the keyed union with the fusion tournament on, ON is `--reflective-merge`. Self-verification on in both. 2,190 calls, 4.6 h model time
RQ4, width 8 (Section 8.4)	`GLM-5.2`	`bbh-keyed`: BBH `dyck_languages` crossed with `KeyedRules`; 60 train / 30 val / 30 test	serial / fork / merge, N=8	32 rollouts per arm (4 rounds); seeds 0–5	Keyed union (`DefaultFusion`) with the fusion tournament and self-verification on; *not* reflective merge — `ReflectiveFusion` records every trial unranked, so the contested counts reported below could not exist under it. 16,124 calls, 28.3 h model time
RQ4, width 4 (Section 8.4)	`GLM-5.2`	as above, same splits	serial / fork / merge, N=4	96 rollouts per arm (24 rounds); seeds 0–5	As above. The width matched to the trust region, at a 3× budget so the round count rises rather than falls

Table 3: The reflective-merge ablation on a one-key artifact. Identical budget, width, seeds and splits; one flag differs. *Fused* counts merges that produced a union at all. *Test* is a split the gate never sees. ON reports no fusion win rate by construction: `ReflectiveFusion` sends its union straight to the gate without ranking it against the singles, so a win rate there would be an artifact rather than a measurement.

	fused / merges	model calls	calls vs. OFF	test (mean)
OFF — keyed union only	**0 / 48**	1001	—	0.250
ON — reflective merge	**42 / 48**	**528**	−47%	0.234

Four seeds, 48 rollouts pinned per seed, N=4, `GLM-5.2`. Calls are per-seed means.

Table 4: Merge-of-N against best-of-N selection and a serial control at an equal rollout budget, on a workload built to satisfy Eq. (6), in two configurations. *Test* is a split no gate in any arm ever saw; *s.d.* is across seeds. *Oracle* is the best fork *on test*, an upper bound that requires the answer to select, against which the fork row reports what fork-and-select can actually ship. *Calls* is the per-seed median. Wall-clock is deliberately not tabulated: the fork arm ran its N sub-runs sequentially, so its recorded wall-clock is a sum rather than what N concurrent forks would take, and the arms of a seed shared one endpoint. Model calls are unaffected by both and are the cost currency we report.

	arm	test min/med/max	mean	s.d.	oracle	calls
N=8, 32	serial	0.133 / **0.267** / 0.333	**0.250**	0.084	—	772
	fork-of-8	0.067 / 0.200 / 0.300	0.189	0.093	0.267	1004
	merge-of-8	0.033 / 0.217 / 0.400	0.194	0.142	—	**685**
N=4, 96	serial	0.100 / 0.250 / 0.433	0.256	0.129	—	2070
	fork-of-4	0.133 / **0.300** / 0.400	**0.283**	0.115	0.333	2363
	merge-of-4	0.200 / 0.267 / 0.400	0.267	**0.073**	—	**1832**

Both blocks: six seeds. Upper: 4 rounds per run. Lower: 24 rounds per run.